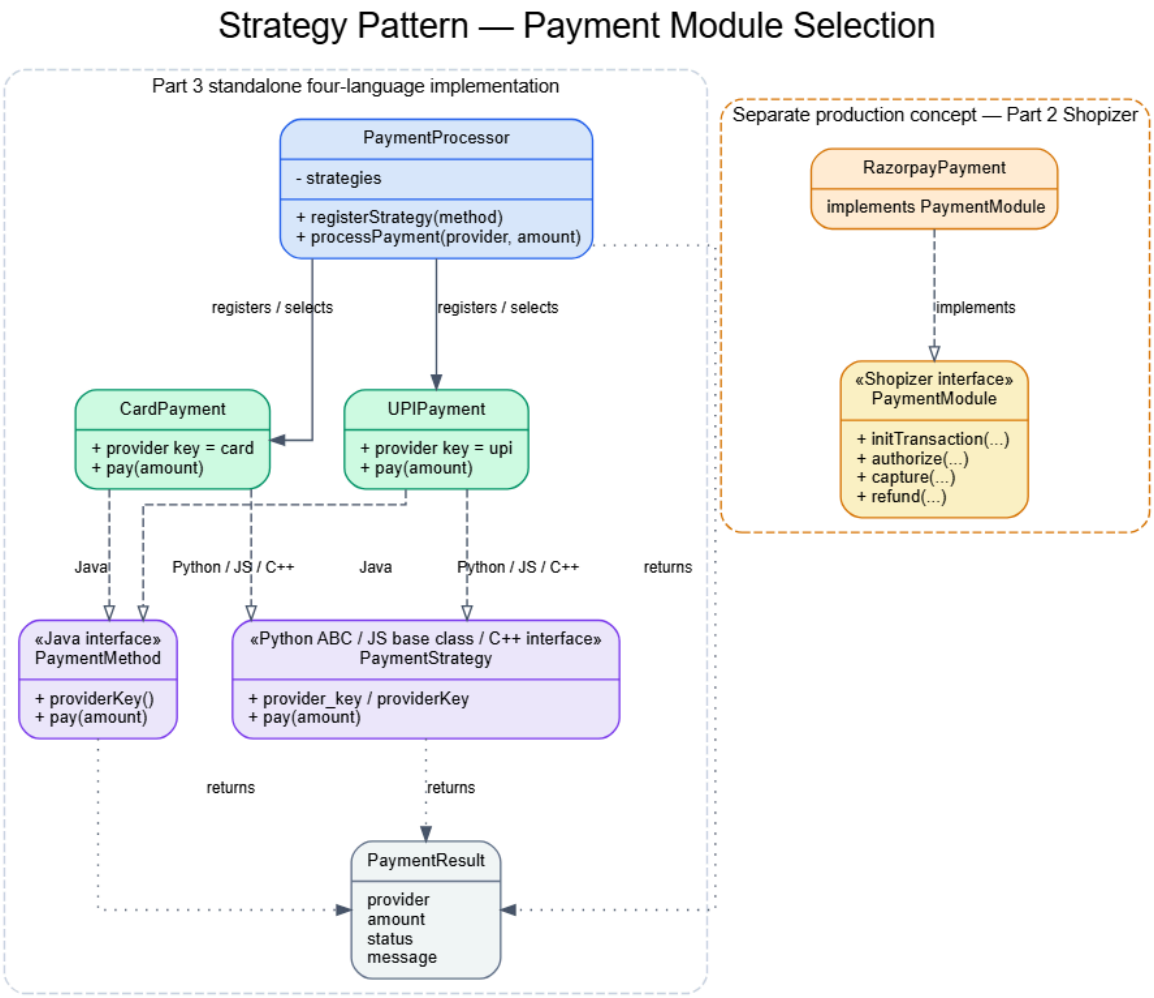
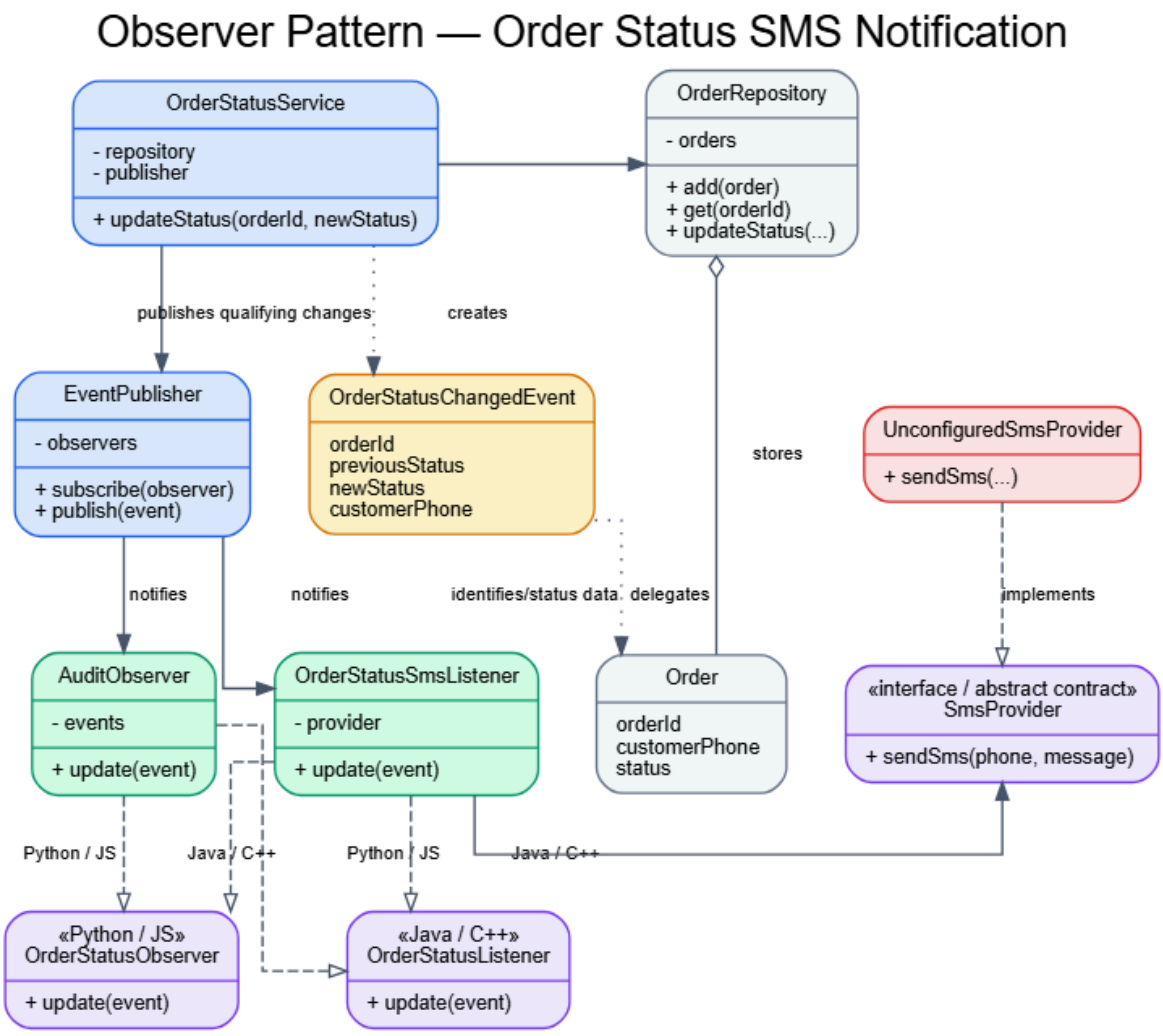
Correcting the UML class diagrams (Output 5)

Review the Strategy and Observer class diagrams for our Shopizer design pattern case study. Compare them with the Java, Python, JavaScript, and C++ implementations and correct any differences in class names, interfaces, methods, or relationships. Make sure the Java PaymentMethod interface is represented correctly, and distinguish the standalone CardPayment and UPIPayment classes from the Razorpay implementation in Shopizer. For the Observer pattern, show the responsibilities of the order status service, event publisher, SMS listener, and SMS provider clearly. Keep the diagrams consistent with the actual code and provide editable Mermaid files along with the rendered diagrams.





Documenting the three evolution scenarios (Output 7)

I need to complete the software evolution section using the original and modified Shopizer 3.2.7 source code. Analyse the three changes: adding Razorpay payments, validating mandatory warranty attributes for Electronics products, and sending SMS notifications when order status changes. For each change, explain what existed before, what was changed, which classes were affected, and how the design pattern was affected. Use exact source paths and verify the relevant code before assigning classifications such as extended, modified, preserved, or newly introduced. Also mention the limits of the testing, especially if actual payment processing or SMS delivery was not performed.

Output 1 — Software evolution analysis (Output 7)

1. Razorpay payment integration (CH01)

Requirement: Add Razorpay as an additional payment option without redesigning Shopizer's existing payment-selection mechanism.

Original implementation

The original application uses the `PaymentModule` interface to define the payment integration contract. `PaymentServiceImpl` selects a provider through a configured map, allowing payment implementations to be selected by key rather than adding provider-specific conditionals throughout the service layer.

Evidence:

- `sm-core/src/main/java/com/salesmanager/core/business/services/payments/PaymentServiceImpl.java`
- `sm-core/src/main/resources/spring/shopizer-core-modules.xml`, lines 45–54 in the original archive.

Design pattern impact: Strategy — Extended

The existing Strategy structure remains in place: `PaymentModule` defines the contract, `PaymentServiceImp` selects the implementation, and the configuration registers the available providers. Razorpay adds another implementation without requiring a replacement of the Strategy abstraction.

Testing limitation

The modified archive contains `RazorpayPaymentTest` with 9 tests, 0 failures, 0 errors, and 0 skipped tests i the recorded Surefire report. The test class uses mocked HTTP interactions. This supports the tested code paths but does not prove that a live Razorpay transaction was completed.

2. Electronics warranty validation (CH02)

Requirement: Require appropriate warranty information for products in the Electronics category.

Original implementation

The original archive does not contain `ElectronicsProductValidator.java`. The new category-specific validator is therefore an evolution addition rather than an unchanged baseline component.

Modified implementation

The modified application introduces:

`sm-shop/src/main/java/com/salesmanager/shop/validation/ElectronicsProductValidator.java`

\

Overall evolution summary

Change	Main affected component	Pattern classification
CH01 — Razorpay	`RazorpayPayment`, payment module XML registry	Strategy extended
CH02 — Electronics warranty	`ElectronicsProductValidator` and catalog validation call sites	Business validation; no new GoF instance identified
CH03 — Order-status SMS	`OrderFacadeImpl`, `OrderStatusChangedEvent`, `OrderStatusSmsListener`	New Observer instance; Facade modified

The recorded focused test reports total 22 tests across these three test classes (9 + 7 + 6), all passing in the supplied reports. These are focused tests and should not be described as proof of live payment processing or SMS delivery.

Comparing the original and modified application (Output 8)

Compare the original Shopizer repository with the modified version and prepare a clear before-and-after design pattern analysis. Verify the payment provider list and count the concrete Strategy implementations, pattern instances, distinct pattern types, participant roles, and unique fully qualified class names separately. Include the existing shipping Strategy, product-search Observer, order-processing Facade, security Builder, and CMS image-manager Singleton, as well as the new order-status SMS Observer. Explain which patterns were preserved, modified, extended, or introduced, and support the findings with source-code evidence. Please recalculate the totals instead of copying numbers from the existing report.

1. Pattern inventory

The source analysis identifies six confirmed GoF pattern instances in the original application and one additional instance after the three evolution changes.

Pattern instance	Baseline	Modified version	Evolution status
Payment module selection	Present	Razorpay added to the payment module registry	Extended
Shipping quote selection	Present	Existing shipping selection retained	Preserved
Product events to search indexing	Present	Existing product event flow retained	Preserved
Order processing coordination	Present	Facade participates in status-change event publication	Modified
Security group construction	Present	Existing builder retained	Preserved
CMS local image file manager	Present	Existing lazy Singleton retained	Preserved
Order-status SMS notification	Not present	New status-change event and SMS listener	Newly introduced

2. Payment Strategy: provider counts must be distinguished

The original and modified XML registries are located at:

`sm-core/src/main/resources/spring/shopizer-core-modules.xml`

The original `paymentModules` map contains six entries. The modified map adds:

XML

```
<beans:entry key="razorpay" value-ref="razorpay" />
```

This produces a registry count of **6 before and 7 after**.

However, counting all concrete payment implementation classes is a different measurement from counting configured registry entries. For example, `PayPalRestPayment` may exist as a class without appearing in the displayed XML map. Any final provider inventory should therefore state which definition of "provider count" is being used and verify the class inventory separately.

Preparing the required Excel and CSV files (Output 12)

Check the assignment template and prepare the required Excel workbook and CSV datasets for the design pattern case study. Include the required files cross_language.csv, pattern_counts.csv, patterns.csv, and llm.csv, along with any supporting comparison or complexity datasets needed. Use the four-language Strategy and Observer implementations for the comparison, and keep the pattern inventory consistent with the original and modified Shopizer analysis. Recalculate the nonblank, non-comment lines of code, verify test totals and complexity values from the available evidence, and make sure the Excel sheets and CSV files contain matching values. Follow the required filenames and column headings from the template.

EXPECTED OUTPUT

- cross_language.csv — language and pattern comparison
- pattern_counts.csv — before/after pattern and role counts
- patterns.csv — pattern inventory and source mapping
- llm.csv — prompt stages and verification manifest
- Workbook with matching sheets and metrics

1. Required files

The dataset package should include the following canonical files:

Filename	Purpose	Required contents
`cross_language.csv`	Cross-language comparison	Language, pattern, implementation characteristics, LOC, tests, and complexity metrics as defined by the assignment template
`pattern_counts.csv`	Before/after pattern counts	Baseline and modified instance counts, pattern types, participant roles, and unique class symbols
`patterns.csv`	Pattern inventory	Pattern name, category, instance, participants, source mapping, baseline status, and post-evolution status
`llm.csv`	LLM usage and verification record	Prompt stage, task, model information where available, output artefact, and human verification status
`cross_language_comparison.xlsx`	Consolidated workbook	Comparison and inventory sheets that agree with the canonical CSV data

Supporting files such as `complexity_comparison.csv`, `language_feature_matrix.csv`, and `pattern_loc_breakdown.csv` may also be included if they provide useful detail and do not conflict with the assignment's required filenames.

The exact column headings should follow the supplied assignment template rather than being invented independently.

Checking the complexity and testing results

Audit the recorded results for the Java, Python, JavaScript, and C++ implementations of Strategy and Observer. Check the test commands and outputs, production LOC calculations, development and verification environments, and maximum cyclomatic complexity for the reported functions. Confirm how complexity was calculated in each language and identify any differences in counting methods. Do not mark a test as passed or a measurement as verified unless the available execution evidence supports it. List any discrepancies that need correction before the results are included in the final workbook.

1. Cross-language test results

The previously recorded standalone implementation test results are:

Language	Strategy tests	Observer tests	Total
Python	10 passed	10 passed	20
JavaScript	10 passed	10 passed	20
Java	10 passed	10 passed	20
C++	10 passed	10 passed	20
Total	**40 passed**	**40 passed**	**80**

2. Part 2 Shopizer test evidence

The modified Shopizer archive contains these recorded Surefire results:

Test suite	Tests run	Failures	Errors	Skipped
`RazorpayPaymentTest`	9	0	0	0
`ElectronicsProductValidatorTest`	7	0	0	0
`OrderStatusSmsListenerTest`	6	0	0	0
`OrderStatusTransitionTest`	2	0	0	0
Total in these four reports	**24**	**0**	**0**	**0**

3. LOC measurement

Use the following recorded values as the current reference, then rerun the scripts if the source files have changed.

Language	Strategy LOC	Observer LOC	Combined LOC
Python	80	151	231
JavaScript	107	201	308
Java	101	234	335
C++	98	170	268

Final consistency check and packaging

Please review the latest versions of the UML diagrams, evolution scenarios, before-and-after analysis, Excel workbook, and CSV files together. Check that pattern

names, class names, provider counts, source paths, LOC values, test totals, and classifications agree across all deliverables. Confirm that the required files are present, the diagram sources are editable, and the documentation explains how to regenerate the datasets. Correct only the issues supported by the available evidence, then provide a concise list of the changes made and any remaining limitations. Do not claim that the package has been fully verified until the checks have actually been completed.